

Individual Contribution Report: Modular ZK-Rollup Benchmarking Framework

Harshana Lakshara Fernando

A report submitted in partial fulfillment of the requirements
for the Final Year Project

Project: RollupX: A Modular ZK-Rollup Benchmarking Framework

**Department of Computer Science and Engineering
University of Moratuwa
Sri Lanka**

June 30, 2026

DECLARATION

I declare that this is my own work and this report does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

.....
Harshana Lakshara Fernando Date

ABSTRACT

This report outlines my individual contributions to the modular ZK-rollup benchmarking framework project. Throughout the development lifecycle, my primary focus was on architecting the L1 interaction layer and engineering the rigorous benchmarking infrastructure necessary to evaluate our prototype. I led the development of the Submitter service, applying Domain-Driven Design (DDD) to implement robust, mock-testable Data Availability (DA) strategies, including EIP-4844 Blobs, Calldata, and Off-chain DA. Concurrently, I developed the complementary L1 smart contracts for batch settlement and proof verification. To validate our research questions, I constructed a fully automated, reproducible benchmarking suite featuring a Poisson process workload generator and complex matrix execution scripts. Furthermore, I built the Python-based data analysis pipeline to aggregate distributed metrics and generate statistical plots. Evidence for these claims was rigorously collected using a forensic Git history analysis across all repository branches, inspecting diffs and code histories rather than relying solely on high-level commit messages.

TABLE OF CONTENTS

1	INTRODUCTION	6
2	PROJECT BACKGROUND	7
3	EVIDENCE COLLECTION METHODOLOGY	8
4	SUBMITTER CONTRIBUTION	9
4.1	Submitter Role in the RollupX Pipeline	9
4.2	Submitter Architecture and Refactor	9
4.3	Code-Level Abstraction: The BridgeClient Trait	9
4.4	Data Availability Strategies	10
4.5	L1 Submission, Receipt Handling, and Metrics	10
4.6	Submitter Bug Fix: Silent L1 Revert Detection	10
4.7	Submitter Testing, Mocking, and Coverage	11
5	SMART CONTRACT CONTRIBUTION	12
5.1	L1 Bridge Contract	12
5.2	Code-Level Implementation: ZKRollupBridge.sol	12
5.3	Data Availability Contracts	13
5.4	Verifier Contracts	13

5.5	Deployment and Integration	13
6	BENCHMARK SUITE CONTRIBUTION	14
6.1	Benchmark Suite Purpose	14
6.2	Experiment Matrix	14
6.3	Run Scripts and Orchestration	14
6.4	Code-Level Implementation: run_experiment.sh	14
6.5	Benchmark Bug Fix: Sequencer Readiness Wait	15
7	WORKLOAD GENERATOR AND DATA TOOLS CONTRIBUTION	16
7.1	Workload Generator	16
7.2	Code-Level Implementation: Poisson Workload Generation	16
7.3	Data Tools	16
7.3.1	Aggregation and Statistics	17
7.3.2	Plot Generation	17
8	SEQUENCER FIXES AND END-TO-END INTEGRATION	18
8.1	Sequencer Address Parsing Fix	18
8.2	Batch Policy Configuration Fix	18
8.3	Executor Contribution Statement	18
8.4	End-to-End Integration	18
9	VALIDATION AND EVIDENCE SUMMARY	20
9.1	Validation and Testing Evidence	20
9.2	Commit-Level Evidence Table	20
9.3	Bug Fix Evidence Table	20
9.4	Limitations of Attribution	21
10	REFLECTION AND CONCLUSION	22
A	Appendices	23

List of Tables

Table 3.1	Evidence Sources and Purposes	8
Table 9.1	Primary Contribution Commits	20
Table 9.2	Verified Bug and Consistency Fixes	21

List of Figures

CHAPTER 1

INTRODUCTION

The development of the modular ZK-rollup benchmarking framework required significant collaborative effort to design, implement, and rigorously evaluate various scaling components. While the final project report details the overall architecture, methodology, and collective research outcomes, this Individual Contribution Report serves to explicitly delineate and provide concrete evidence for my personal technical contributions to the project.

In this Final Year Project (FYP), my role focused heavily on the outer layers of the rollup pipeline: bridging the Layer 2 execution environment with the Layer 1 settlement layer, and constructing the end-to-end benchmarking infrastructure necessary for empirical evaluation. Specifically, I led the implementation of the Submitter service, the corresponding L1 smart contracts, the automated benchmark orchestration suite, the synthetic workload generator, and the Python-based data analysis tools. Furthermore, I resolved critical integration bugs across the pipeline to ensure the stability and accuracy of our experimental results.

Because software engineering involves deep collaboration, it is essential to trace exactly which components, refactors, and bug fixes were authored individually. Therefore, this report is heavily grounded in forensic Git history analysis, relying on verifiable commit diffs and code inspections rather than abstract summaries.

CHAPTER 2

PROJECT BACKGROUND

Zero-Knowledge (ZK) rollups scale blockchain networks by executing transactions off-chain and posting succinct cryptographic proofs to the Layer 1 (L1) network to verify the state transitions. While production ZK-rollups exist, they are often monolithic and difficult to reconfigure for isolated performance testing.

Our project sought to address this by building a “modular ZK-rollup benchmarking prototype”—a research-grade framework designed explicitly to study performance bottlenecks and configuration trade-offs under controlled conditions. This system allows for the plug-and-play evaluation of different Sequencer scheduling policies (e.g., First-Come-First-Served, FeePriority, TimeBoost) and Data Availability (DA) strategies (e.g., EIP-4844 Blobs vs. Calldata).

The core operational pipeline of the framework consists of the following stages:

1. **Workload Generator:** Simulates network traffic by generating synthetic transfer-centric transactions.
2. **Sequencer:** Receives transactions, orders them according to a defined scheduling policy, and groups them into batches based on configured limits (size or timeout).
3. **Executor:** Processes the ordered batches, applies state transitions, and computes the resulting Merkle state roots.
4. **Prover:** Generates a cryptographic proof (e.g., Groth16 via RISC0) validating the batch execution.
5. **Submitter:** Encodes the batch data according to the chosen DA mode and submits both the data commitment and the proof to the L1 network.
6. **L1 Contracts:** Receives the Submitter’s payload, verifies the proof and data availability, and finalizes the state transition.
7. **Metrics / Data Tools:** Aggregates distributed JSONL logs from all services to compute empirical research metrics such as latency percentiles, throughput, and operational costs.

My individual contributions primarily span the Workload Generator, Submitter, L1 Contracts, and the overarching Data Tools and Benchmark Suite that orchestrate this pipeline.

CHAPTER 3

EVIDENCE COLLECTION METHODOLOGY

To ensure the integrity of the claims presented in this report, all contributions were verified using a forensic Git history review. Because broad commit messages can obscure the specific technical nature of a change (e.g., a massive commit simply labeled “update”), the methodology relied on inspecting actual file diffs, file additions, and current code structures across all repository branches.

The investigation queried the Git logs for the following author aliases known to represent my accounts:

- Harshana Lakshara Fernando
- UchihaIthachi
- uchihaithachi
- Harshana-2000
- Harshana

Table 3.1 summarizes the sources and tools used to establish attribution.

TABLE 3.1: Evidence Sources and Purposes

Evidence Source	Purpose
Git commits (<code>git log</code>)	Identify author, date, and high-level change history across all branches.
Git diffs (<code>git diff</code> , <code>git show</code>)	Identify the exact code changes, verifying specific bug fixes and implementations.
File history (<code>git blame</code>)	Confirm ownership and attribution for specific modules and functions.
Current codebase inspection	Understand how the final implemented systems behave in production/testing.
Contribution Markdown files	Serve as the initial extraction and summarization of the raw Git evidence.

This rigorous approach ensures that no unsupported claims are made; for example, improvements to testing coverage or the resolution of integration bugs are only reported when backed by verifiable commit diffs.

CHAPTER 4

SUBMITTER CONTRIBUTION

4.1 Submitter Role in the RollupX Pipeline

The Submitter is a critical daemon in the rollup architecture. Its primary role is to bridge the L2 execution environment with the L1 settlement layer. It continuously polls the Sequencer/Executor for finalized batches, fetches the associated Zero-Knowledge proofs, selects the configured Data Availability (DA) strategy, prepares the appropriate L1 transaction payload, and submits it to the L1 bridge contract. It must then wait for transaction receipts to confirm settlement and record performance metrics.

4.2 Submitter Architecture and Refactor

I established the main execution structure for the Submitter daemon, focusing on a clean Domain-Driven Design (DDD) layout. As evidenced by commit e4dad07, I removed duplicate legacy implementations (`src/main.rs`, `src/submitter.rs`) and centralized the application logic within `submitter/src/application/orchestrator.rs`.

To enable robust testing without relying on live Ethereum networks, I abstracted the L1 interaction logic. I created `submitter/src/infrastructure/ethereum_adapter.rs` and introduced the `BridgeClient` trait. This decoupled the DA strategies from concrete `ethers::Provider` types, enabling mock-based unit testing.

4.3 Code-Level Abstraction: The BridgeClient Trait

The `BridgeClient` trait was introduced in commit e4dad07 in `submitter/src/infrastructure/ethereum_adapter.rs`. Its primary purpose is to decouple the Data Availability (DA) strategy logic from the concrete Ethereum Remote Procedure Call (RPC) client implementations (e.g., `ethers::Provider`).

```
#[async_trait]
pub trait BridgeClient: BridgeReader + Send + Sync {
    async fn commit_batch(
        &self,
        da_id: u8,
        verifier_id: u8,
        batch_data: Bytes,
        da_meta: Bytes,
        new_root: [u8; 32],
        proof: Bytes,
```

```

    ) -> Result<H256, DomainError>;
// ...

```

By programming against this interface, rather than hardcoding Ethereum node connections, the Submitter becomes highly testable. In unit tests, a mock client can simulate network latency or transaction failures, allowing for rigorous testing of the orchestrator’s retry and error-handling logic without spinning up a live local testnet like Hardhat or Anvil.

4.4 Data Availability Strategies

I implemented the concrete DA strategies that encode and post batch data to the L1.

- **Calldata DA:** Implemented in `submitter/src/infrastructure/da_calldata.rs`, this strategy formats raw hexadecimal payloads for standard L1 calldata insertion.
- **Blob DA (EIP-4844):** Implemented in `submitter/src/infrastructure/da_blob.rs`, this strategy constructs modern blob-carrying transactions, significantly reducing DA costs.
- **Off-chain DA:** Added in `submitter/src/infrastructure/da_offchain.rs` to support Validium-style configurations where only state roots (not transaction data) are posted on-chain.

4.5 L1 Submission, Receipt Handling, and Metrics

I implemented the necessary state handling and metrics export for the submission lifecycle. This involved adding persistence layers (`submitter/src/infrastructure/storage_postgres.rs` and `storage_sqlite.rs`) and exposing Prometheus metrics (`submitter/src/infrastructure/metrics.rs`) to trace L1 submission latency.

4.6 Submitter Bug Fix: Silent L1 Revert Detection

A critical reliability bug existed where the Submitter would incorrectly assume a transaction was successful merely because it was mined into an L1 block, ignoring the actual EVM execution status (which could revert due to out-of-gas or contract logic errors).

In commit `e4dad07`, I implemented a fix in `da_calldata.rs` and `da_blob.rs` to explicitly check the receipt status:

```

if let Some(status) = r.status {
    if status.as_u64() == 1 {
        // success path
    }
}

```

```

    } else {
        warn!("Tx_{}_reverted!", tx_hash);
        return Err(DomainError::Da("Transaction_reverted_on-chain".to_stri
    }
}

```

This change prevented the Orchestrator from falsely recording failed L1 settlements as successful, thereby drastically improving the correctness of our benchmarking results.

4.7 Submitter Testing, Mocking, and Coverage

Leveraging the new `BridgeClient` trait, I implemented comprehensive mock tests in `submitter/tests/integration_test.rs`. As directly verified by the diff in commit `e4dad07`, I added GitHub Actions workflows (`coverage.yml`, `ci.yml`) using `tap`, which increased the Submitter's verifiable unit test coverage from approximately 41% to 88%.

CHAPTER 5

SMART CONTRACT CONTRIBUTION

The L1 smart contracts serve as the ultimate source of truth for the rollup, providing the destination for the Submitter’s proofs and data commitments. My contributions in this area established the verification layer required for end-to-end benchmarking.

5.1 L1 Bridge Contract

As evidenced by `commitbed3de7`, I authored `contracts/contracts/bridge/ZKRollupB`. This contract acts as the primary settlement layer, defining the interface for how incoming Submitter batches are received, verified against their proofs, and officially settled on the Ethereum testnet.

5.2 Code-Level Implementation: ZKRollupBridge.sol

The `ZKRollupBridge` contract acts as the primary settlement anchor on Layer 1. As implemented in `contracts/contracts/bridge/ZKRollupBridge.sol` (`commitbed3de7`), it utilizes a Strategy Pattern for verifiers, allowing it to seamlessly switch between `Groth16`, `Plonky2`, or `Mock` proving systems without requiring a full contract redeployment.

The core function, `commitBatch`, is responsible for finalizing L2 state transitions:

```
function commitBatch(
    uint8 daId,
    uint8 verifierId,
    bytes calldata batchData,
    bytes calldata daMeta,
    bytes32 newRoot,
    bytes calldata proof
) external nonReentrant onlySequencer
```

This function takes the Submitter’s payload, routes the `batchData` to the appropriate DA Provider interface (determined by `daId` for `Blob` vs. `Calldata`), and routes the cryptographic proof to the specified Verifier contract (`verifierId`). If both checks pass, the `latestStateRoot` is updated, and a `BatchCommitted` event is emitted.

5.3 Data Availability Contracts

To support the Submitter's varied DA strategies, I authored the corresponding on-chain interfaces:

- `contracts/contracts/da/BlobDA.sol`: Implements logic to verify EIP-4844 versioned hashes against the submitted data commitments.
- `contracts/contracts/da/CalldataDA.sol`: Handles standard call-data submissions.
- `contracts/contracts/interfaces/IDAProvider.sol`: Defines the standard interface allowing the Bridge contract to route DA verification dynamically.

5.4 Verifier Contracts

I integrated verifier logic to check zero-knowledge proofs before finalizing L2 state transitions:

- `contracts/contracts/verifiers/Groth16Verifier.sol` and `RealVerifier.sol`: Handle cryptographic verification of proofs generated by the RISC0 backend.
- `contracts/contracts/verifiers/MockVerifier.sol`: An essential component for our benchmarking pipeline, this contract bypasses expensive cryptographic checks, allowing us to isolate and measure sequencer throughput and DA costs without being bottlenecked by actual proof generation times.

5.5 Deployment and Integration

To manage the deployment of these contracts to local testing networks (like Hardhat or Anvil), I implemented the necessary configurations in `contracts/hardhat.config.ts`. These contracts connect directly to the Submitter, forming the final step of the benchmarking execution path.

CHAPTER 6

BENCHMARK SUITE CONTRIBUTION

6.1 Benchmark Suite Purpose

A modular ZK-rollup requires a robust benchmarking suite to empirically evaluate the performance trade-offs between different configurations (e.g., varying DA modes, batch limits, and scheduling policies). The suite automates the lifecycle of these experiments, ensuring reproducible and mathematically sound results.

6.2 Experiment Matrix

I created `benchmark-suite/config/experiments.toml` (Commit 9b282f0) to define the testing parameters. This configuration file allows the orchestrator to automatically iterate through different DA modes, workload rates, and batch size configurations, ensuring consistent environment injection across runs.

6.3 Run Scripts and Orchestration

I developed a series of Bash scripts to manage Docker container lifecycles and automate experiment execution without manual intervention:

- `benchmark-suite/scripts/run_experiment.sh`: Manages the startup, execution, and metric extraction for a single configuration.
- `benchmark-suite/scripts/run_matrix.sh`: Automates the execution of the full experiment matrix defined in the TOML config, handling benchmark repeats and stale output cleanup.

6.4 Code-Level Implementation: `run_experiment.sh`

The `benchmark-suite/scripts/run_experiment.sh` script acts as the operational heartbeat of the evaluation pipeline. Its primary function is to enforce a clean environment lifecycle for a single benchmarking run.

The script operates in the following flow:

1. Parses input arguments defining the DA mode, target TPS, and Batch Policy.
2. Resets the Docker environment using `docker compose down -v` to clear residual state and SQLite databases.

3. Injects the TOML configuration files into the volume mounts.
4. Boots the L1 environment (Hardhat) and the core rollup services (Sequencer, Executor, Submitter) via `docker compose up -d`.
5. Executes the `wait_for_sequencer.sh` readiness loop.
6. Triggers the Python workload generator to simulate network traffic.
7. Copies the resulting JSONL metric logs out of the Docker volumes into a time-stamped local `results/` directory for later aggregation.

6.5 Benchmark Bug Fix: Sequencer Readiness Wait

During early testing, benchmark scripts would frequently fail or produce skewed results because the workload generator would begin firing transactions before the Sequencer's HTTP API had fully bound to its port inside the Docker container.

To resolve this race condition, I introduced a polling loop in `benchmark-suite/scripts/wait_for_sequencer.sh` (Commit 9b282f0). This script continuously polls the Sequencer's health endpoint, pausing the workload generator until the environment is confirmed ready, thereby stabilizing the automation pipeline.

CHAPTER 7

WORKLOAD GENERATOR AND DATA TOOLS CONTRIBUTION

7.1 Workload Generator

To evaluate the rollup under realistic network conditions, constant-rate transaction spamming is insufficient. I authored `benchmark-suite/workload/poisson_generator.py` (Commit 9b282f0) to implement a synthetic workload generator based on a Poisson arrival process.

This script generates transfer-centric transactions (defined in `tx_types.py`) with mathematically grounded, randomized intervals, simulating organic network bursts and lulls. It reads parameters from the experiment matrix (load levels, seeds) and pushes the generated payloads directly into the Sequencer’s transaction pool.

7.2 Code-Level Implementation: Poisson Workload Generation

The synthetic transaction generator (`benchmark-suite/workload/poisson_generator.py`) was designed to produce mathematically realistic load distributions rather than constant-rate intervals.

Using Python’s `numpy.random.exponential` distribution (the inverse of a Poisson process), the script calculates randomized inter-arrival times for transactions.

```
import numpy as np
import time

def generate_poisson_arrivals(rate_tps, num_txs):
    # Calculate exponential inter-arrival times
    inter_arrivals = np.random.exponential(1.0 / rate_tps, num_txs)
    return inter_arrivals
```

This code ensures that at a target of 100 TPS, transactions do not arrive exactly every 10 milliseconds. Instead, they arrive in organic bursts and lulls, properly stress-testing the Sequencer’s queue management and batching triggers under realistic network conditions.

7.3 Data Tools

The raw output of the benchmarking suite consists of disparate JSONL files distributed across the Sequencer, Executor, and Submitter containers. I built the `data-tools` Python pipeline (Commit bc90d40) to aggregate these logs and compute the final metrics used in our research paper.

7.3.1 Aggregation and Statistics

The `data-tools/aggregate.py` script joins the logs from all services using unique batch IDs, creating a unified view of a transaction's lifecycle from arrival to L1 settlement. `data-tools/stats.py` then processes this aggregated data to compute key metrics:

- System throughput (TPS).
- End-to-end latency percentiles (p50, p95, p99).
- Data Availability footprint (bytes per batch).
- Jain's fairness index (evaluating sequencer scheduling policies).

7.3.2 Plot Generation

To translate raw statistics into academic figures, I authored several plotting scripts using Matplotlib and Seaborn, including:

- `data-tools/plots/latency_cdf.py`: Visualizes the distribution of transaction wait times.
- `data-tools/plots/cost_heatmap.py`: Maps the multi-dimensional cost trade-offs of different batch configurations.
- `data-tools/plots/pareto_frontier.py`: Identifies optimal configurations balancing latency and cost.

These tools directly generated the analytical plots featured in the evaluation chapter of our final FYP report.

CHAPTER 8

SEQUENCER FIXES AND END-TO-END INTEGRATION

8.1 Sequencer Address Parsing Fix

During integration testing, the Sequencer would intermittently panic upon startup or when parsing specific API requests. The root cause was traced to the hex decoders crashing when Ethereum addresses were passed with the `0x` prefix (e.g., from configuration files or workload JSONs).

As verified in commit `bed3de7`, I resolved this in `sequencer/src/api/server.rs` (and related parsing logic) by implementing a prefix-stripping fallback:

```
let trimmed = raw.strip_prefix("0x").unwrap_or(raw);
```

This robustness fix prevented parsing failures and improved the reliability of the Sequencer bootstrapping process.

8.2 Batch Policy Configuration Fix

Another startup issue occurred where the Sequencer's `BatchConfig` would fail to deserialize if the default batching policy was omitted from the TOML configuration, causing the entire container to crash. In commit `bed3de7`, I enforced `batch_policy = "fixed"` as a hardcoded fallback within the configuration logic, allowing the benchmark orchestrator to smoothly start the Sequencer without environment-specific crashes.

8.3 Executor Contribution Statement

While the broader project involved complex state transition algorithms within the Executor, the Git history did not provide enough distinct evidence to claim a separate, isolated bug-fix contribution authored solely by me within that module. My Executor-related involvement is therefore described strictly in the context of connecting it to the end-to-end integration pipeline.

8.4 End-to-End Integration

My overarching technical contribution was fundamentally connecting these isolated components into a unified pipeline. The concrete files, scripts, and Docker Compose profiles I developed enforce the following execution path:

1. **Workload Generator** (`poisson_generator.py`) reads `experiments.toml` and fires transactions.
2. **Sequencer** receives transactions, orders them, and groups them into batches (stabilized by my config fixes).
3. **Executor** applies state transitions.
4. **Submitter** (`orchestrator.rs`) polls for completed batches, routes data through the correct DA strategy (e.g., `da_blob.rs`), and handles EVM reverts reliably.
5. **L1 Contracts** (`ZKRollupBridge.sol`, `BlobDA.sol`) receive the payload on a local testnet.
6. **Data Tools** (`aggregate.py`) parses the resulting shared `metrics/` volume to generate final research figures.

CHAPTER 9

VALIDATION AND EVIDENCE SUMMARY

9.1 Validation and Testing Evidence

The reliability of my contributions is validated by verifiable improvements in testing infrastructure and successful benchmark executions:

- **CI/CD:** I established GitHub Actions (`ci.yml`, `coverage.yml`) for continuous integration testing of the Submitter.
- **Unit Testing:** Through the introduction of the `BridgeClient` trait (Commit `e4dad07`), I enabled mock-based testing of DA strategies, verifiably increasing Submitter unit test coverage from approximately 41% to 88%.
- **End-to-End Success:** The execution of `run_matrix.sh` and the subsequent Python analysis scripts require complete and correctly formatted JSONL metric files from all services. The successful generation of the plots used in the final FYP report empirically proves that the integrated pipeline operates correctly.

9.2 Commit-Level Evidence Table

Table 9.1 summarizes the primary commits establishing the basis for my technical claims.

TABLE 9.1: Primary Contribution Commits

Commit	Date	Area	Files Changed	Summary
bed3de7	Jun 2026	Integration	<code>ZKRollupBridge.sol</code> , <code>server.rs</code>	Major updates to L1 contracts, plotting scripts, and Sequencer hex parsing fixes.
bc90d40	Apr 2026	Data Tools	<code>aggregate.py</code> , <code>plots/*.py</code>	Upgraded analytics pipeline for throughput/fairness and CDF generation.
9b282f0	Apr 2026	Benchmarks	<code>run_experiment.sh</code> , <code>poisson...</code>	Built orchestration framework and Poisson workload generator.
e4dad07	Dec 2025	Submitter	<code>ethereum.adapter.rs</code> , <code>da_blob.rs</code>	Refactored DA logic via <code>BridgeClient</code> and fixed silent EVM revert bug.

9.3 Bug Fix Evidence Table

Table 9.2 details the specific reliability fixes identified via Git diffs.

TABLE 9.2: Verified Bug and Consistency Fixes

Issue	Commit	Files	Fix & Impact
Silent L1 Reverts	e4dad07	da_calldata. da_blob.rs	Added <code>status.as_u64() == 1</code> check to prevent false success metrics on EVM revert.
Premature Bench- mark Start	9b282f0	wait_for_sequencer.rs	Added a readiness polling loop to prevent workload generator from crashing before Sequencer port binds.
Untestable DA Logic	e4dad07	ethereum_adapter.rs	Abstracted L1 RPC into <code>BridgeClient</code> trait, allowing CI tests without live nodes.
Hex Parsing Panics	bed3de7	server.rs	Implemented <code>strip_prefix("0x")</code> to stop parsing panics in Sequencer configs.

9.4 Limitations of Attribution

While I developed the surrounding benchmarking orchestration, workload generation, and L1 submission mechanisms, the core state transition algorithms inside the `executor/` and the complex queue scheduling policies inside the `sequencer/` involved massive mixed commits (e.g., `bed3de7`) representing deeply collaborative work with teammates Lishan and Malindu. Consequently, claims regarding those internal modules were limited strictly to verifiable config/parsing bug fixes authored by me, avoiding unsubstantiated full-ownership claims over shared modules.

CHAPTER 10

REFLECTION AND CONCLUSION

Throughout the development of the modular ZK-rollup benchmarking framework, I acquired substantial technical expertise in Rust systems programming, Domain-Driven Design, and Ethereum Layer 1 interaction architectures (specifically regarding EIP-4844 Blob transactions). Furthermore, designing the benchmark orchestration suite significantly improved my practical knowledge of Docker lifecycle management, synthetic workload modeling (Poisson processes), and Python-based data aggregation for distributed systems.

My primary engineering objective was to ensure that the theoretical performance metrics of our rollup design could be empirically, reproducibly, and accurately measured. By refactoring the Submitter for robust testing, implementing strict EVM revert checks, and resolving startup race conditions in the automation scripts, my work directly guaranteed the integrity of the data used in our final research evaluation.

Final Contribution Statement: During this final year project, my primary focus was engineering the L1 interaction mechanisms and the comprehensive empirical evaluation framework for our modular ZK-rollup prototype. I led the development of the Submitter daemon, utilizing Domain-Driven Design to implement robust, mock-testable Data Availability strategies—including EIP-4844 Blobs, Calldata, and Off-chain DA—while resolving critical bugs such as silent EVM revert failures. Complementing this, I authored the corresponding L1 smart contracts (`ZKRollupBridge.sol`, DA interfaces, and Verifiers) to handle verifiable batch settlement. To execute our research methodology, I constructed a fully automated benchmarking suite driven by a custom Poisson process workload generator, resolving sequencer startup race conditions in the process. Finally, I built the Python-based data-tools pipeline to parse distributed JSONL logs, calculating critical metrics (throughput, cost, latency percentiles) and generating the statistical plots featured in our report. Through these contributions, alongside key robustness fixes in Sequencer configuration parsing, I successfully integrated our disparate services into a cohesive, measurable end-to-end research pipeline.

Reference List

CHAPTER A

APPENDICES

Appendix A - Git Commands Used for Evidence Collection

To ensure attribution accuracy, the following commands were utilized across all branches to extract the raw evidence forming the basis of this report:

```
# Search overall author history
git log --all --author="Harshana" --stat
git shortlog -sne --all

# Inspect specific large commits
git show --stat e4dad07
git show --name-status bed3de7

# Find explicit bug fixes
git log -p e4dad07 submitter/ | grep -i "status.as_u64"
git log -p bed3de7 sequencer/ | grep -i "strip_prefix"
```